# Level 2

Firstly we need that when is_nr = 1, isResponded = False and when is_nr = 0, isResponded = True. Executing the below code, I found that df_misaligned was empty. The top level variables are aligned fine.

**Original Code**

```
# load the data using pandas
import pandas as pd
import numpy as np
import json

pd.set_option('display.max_colwidth', None)
df = pd.read_csv("test_attempt_question_responses-202209061938.csv")


rows = []
for row in df.itertuples(index=False):
   response = json.loads(row.response_raw)['3']

   # Naive check: Check that when row.is_nr and response['isResponded'] are always
opposite
   # Result: there are no rows where this is misaligned
   if (row.is_nr == 1 and response['isResponded'] == True) or (row.is_nr == 0 and
response['isResponded'] == False):
       rows.append({
       "id":row.id,
       "test_question_id": row.test_question_id,
       "is_nr": row.is_nr,
       "isResponded": response['isResponded'],
   })
df_misaligned = pd.DataFrame(rows)
```

**But,** then I saw that this is a drag and drop question : ‘moveable_dnd’ as it says in question type. So, I am assuming that the targets mentioned are the places where objects could be dropped and their respective contents are the actual objects dropped in that space.

Thus, I checked whether for each response id (“id”), all the targets have a non-empty content or not. If at least one of the target content is not empty, it means that this entire response is responded. So response[‘isResponded’] must be true for this. So I checked this with the code below and found 20 rows which are misaligned (see on next page).

**New Code**

```
# load the data using pandas
import pandas as pd
import numpy as np
import json

pd.set_option('display.max_colwidth', None)
df = pd.read_csv("test_attempt_question_responses-202209061938.csv")


rows = []
for row in df.itertuples(index=False):
   response = json.loads(row.response_raw)['3']

   # Advanced check: check if each of the targets has atleast a non empty content.
If so, it is responded.
   # Actually gave 20 rows where response['isResponded'] == False eventhough there
are responses
   targets = response.get('targets')
   responded = False # this is the flag which checks if atleast one content is not
empty
   for target in targets:
       if target['contents'] != []:
           responded = True
           break

   if not (responded and response['isResponded']):
       # print('Yoo misaligned')
       rows.append({
       "id":row.id,
       "test_question_id": row.test_question_id,
       "is_nr": row.is_nr,
       "isResponded": response['isResponded'],
   })
df_misaligned = pd.DataFrame(rows)
```

**Df_misaligned data frame.**

| | id | test_question_id | is_nr | isResponded |
|---|---|---|---|---|
| 0 | 12709627 | 13023 | 1 | False |
| 1 | 12709974 | 13023 | 1 | False |
| 2 | 12714414 | 13023 | 1 | False |
| 3 | 12715673 | 13023 | 1 | False |
| 4 | 12717728 | 13023 | 1 | False |
| 5 | 12725370 | 13023 | 1 | False |
| 6 | 12726641 | 13023 | 1 | False |
| 7 | 13278922 | 13023 | 1 | False |
| 8 | 13717153 | 13023 | 1 | False |
| 9 | 13828518 | 13023 | 1 | False |
| 10 | 14043233 | 13023 | 1 | False |
| 11 | 14043402 | 13023 | 1 | False |
| 12 | 14578364 | 13023 | 1 | False |
| 13 | 15297504 | 13023 | 1 | False |
| 14 | 15830483 | 13023 | 1 | False |
| 15 | 15836789 | 13023 | 1 | False |
| 16 | 18777220 | 13023 | 1 | False |
| 17 | 19075097 | 13023 | 1 | False |
| 18 | 20204418 | 13023 | 1 | False |
| 19 | 20246473 | 13023 | 1 | False |

This dataframe is also attached as a csv file in the folder. This shows that there are 20 records which in fact have been responded to, i.e. at least one of the targets in the response has a non-empty content. But they were marked as is_nr= 1 or isResponded = False. These are clearly anomalies.

**Reason**: Perhaps isResponded is present at a UI level which is marked true or false depending on whether the user initiated the responding process and the is_nr flag is informed by isResponded on the database side. This is indicative of the fact that they always correspond as in the first naive check. So these misaligned records are added in between the UI and database data transfer process. So they don't exactly originate from a user interaction.